

Kapitel 5

Zeichen und Strings

5.1 Stringvariablen und –konstanten

Den Datentyp `char` haben wir bereits kennengelernt. Ebenso wurde bereits angesprochen, wie Stringkonstanten aussehen. Wie sieht es mit Stringvariablen aus? Diese werden durch Felder von `char`-Werten realisiert. Ein Feld von Zeichen wird dadurch zu einem String, dass man hinter dem letzten „echten“ Zeichen ein *Null-Zeichen* speichert. Bei Stringkonstanten sorgt der Compiler sogar dafür, dass dieses Null-Zeichen automatisch angehängt wird. Das Null-Zeichen hat die Ordnungszahl 0 und kann im Quelltext als `\0` geschrieben werden.

Damit wissen wir jetzt auch, warum `'w'` und `"w"` nicht dasselbe bedeuten:

- `'w'` hat als Wert die Ordnungszahl des Zeichen `w`.
- `"w"` hat als Wert die Zeichenfolge aus dem Zeichen `w` und einem nachfolgenden Null-Zeichen.

Vereinbart man Stringvariablen, so muss man bei der Längenangabe das Null-Zeichen stets mitzählen. Von den Vereinbarungen

```
char text1[4] = "Text",      /* nicht korrekt */
    text2[5] = "Text",
    text3[6] = "Text";
```

ist die erste also *nicht* korrekt, weil sie das abschließende Null-Zeichen *nicht* berücksichtigt. Bei der zweiten Vereinbarung sind die Länge von Variable und Konstante gerade identisch. Die dritte Vereinbarung ist auch zulässig; bei ihr bleibt die letzte der sechs verfügbaren Zeichenpositionen frei.

Eine zusätzliche Möglichkeit für Stringkonstanten sollte hier noch erwähnt werden, weil sie häufiger nützlich ist: Längere Stringkonstanten kann man, in Teile zerlegt, in mehrere Zeilen schreiben. Dazu beendet man die Teile am Zeilenende mit einem Gänsefüßchen und leitet den nächsten Teil wieder mit einem Gänsefüßchen ein; zwischen den Teilen dürfen neben dem Zeilenende-Zeichen auch Leerzeichen und horizontale Tabulatoren stehen. Der Compiler fügt diese Teile unmittelbar hintereinander. Welche der beiden Darstellungen

```
printf("Dieses ist ein nicht besonders langer String\n");
printf("Dieses ist ein nicht besonders "
      "langer String\n");
```

wir wählen, ist also für das Resultat gleichgültig. Man beachte das Leerzeichen am Ende des ersten Teilstring. Es ist notwendig, da beim Zusammenfügen der Zeichenketten keine zusätzlichen Leerzeichen eingefügt werden.

5.2 Arbeiten mit Strings

Offensichtlich erlaubt das Null-Zeichen zweierlei:

- Das Ende eines Strings lässt sich erkennen, ohne dass man seine Länge vorab kennen muss. Die Funktionen `scanf` und `printf` sind zum Beispiel darauf angewiesen, weil sie die Länge ihres Formatierungsstrings ja nicht mitgeteilt bekommen.
- Ein String kann seine Länge während der Ausführung des Programms dynamisch ändern, da er ohne weiteres kürzer sein darf als die Stringvariable, in der er gespeichert ist.

Dass die Umkehrung strikt verboten ist, dass also ein String (einschließlich des abschließenden Null-Zeichens) die Länge des Feldes nie übersteigen darf, ist klar: Da Stringvariablen letztlich Felder sind, gelten natürlich auch dieselben Regeln wie für Felder. Erneut ist ausschließlich der Programmierer selbst dafür verantwortlich, dass keine Indexüberschreitungen vorkommen.

Auch für das Anhängen des abschließenden Null-Zeichens an einen String ist, abgesehen von den Stringkonstanten und einigen Standardfunktionen, der Programmierer selbst verantwortlich.

Als erstes Beispiel betrachten wir eine Anweisungsfolge, die einen String kopiert, dabei seien `quelle` und `ziel` entsprechende Felder, wobei `ziel` groß genug ist um den zu kopierenden String aufzunehmen.

```
i = 0;
while (quelle[i] != '\0') {
    ziel[i] = quelle[i];
    i++;
}
ziel[i] = '\0';
```

Diese Formulierung ist für C allerdings ziemlich untypisch. In der Regel wird man

```
i = 0;
while (ziel[i] = quelle[i]) {
    i++;
}
```

finden. Das funktioniert, weil das Zeichen `'\0'` gerade die Ordnungszahl Null besitzt. Die Bedingung der `while`-Anweisung wird also gerade dann *falsch*, wenn bei der Wertzuweisung das Null-Zeichen übertragen wurde.

Die Vorteile der zweiten Formulierung gegenüber der ersten sind offensichtlich:

- Die Schleife terminiert erst, wenn das Null-Zeichen bereits übertragen ist, so dass es nicht hinter der Schleife besonders behandelt werden muss.
- In jedem Schleifendurchlauf wird nur zweimal auf eine Vektorkomponente zugegriffen und nicht dreimal.

5.3 Ein-/Ausgabe von Zeichen

Da die Verarbeitung von Zeichen und Strings ein wesentlicher Aspekt von C ist, gibt es in der Standardbibliothek, vermittelt durch `stdio.h`, spezielle Funktionen zu ihrer Ein- und Ausgabe. Wir wollen uns zunächst nur die Funktionen für Zeichen ansehen.

Die Funktion `getchar` () liefert als Funktionswert das nächste Zeichen von der Standardeingabe – oder ggf. den Wert `EOF`. Damit es mit `EOF` keine Probleme gibt, muss der

Funktionswert einen Typ besitzen, in dem sich alle Zeichen und zusätzlich EOF darstellen lassen, und das ist gerade der Typ `int`. Entsprechend muss der Funktionswert bei der Zuweisung an eine Zeichenvariable umgewandelt werden. Nach der Zuweisung ist kein Vergleich mit EOF mehr möglich, da EOF nicht im Wertebereich von `char` liegt.

Die Umkehrung ist die Funktion `putchar (c)`. Sie schreibt das Zeichen `c` auf die Standardausgabe. Dabei ist zu beachten: `c` kann den Typ `int` besitzen oder wird (wenn es den Typ `char` oder `short` besitzt) in den Typ `int` umgewandelt. Die Funktion selbst wandelt diesen Wert in den Typ `unsigned char` zurück um, um das zu schreibende Zeichen zu bestimmen. Der Funktionswert ist das geschriebene Zeichen im Typ `int`, wenn kein Fehler aufgetreten ist, bzw. EOF sonst.

Als Beispiel betrachten wir eine Anweisungsfolge, die die Standardeingabe auf die Standardausgabe kopiert:

```
#include <stdio.h>

int c;
while ((c = getchar ()) != EOF) {
    putchar (c);
}
```

Dass diese Anweisungsfolge (bei der üblichen Zuordnung von Standardein- und -ausgabe) *kein* Durcheinander auf dem Bildschirm erzeugt, hat zwei simple Hintergründe: Zum einen arbeitet `getchar` wie `scanf` gepuffert, d.h. es wird zunächst das ENTER des Benutzers abgewartet und dann die eingegebene Zeile Zeichen für Zeichen abgearbeitet. Zum anderen liefert `getchar` das Zeilenende-Zeichen wie alle anderen Zeichen auch an das Programm ab, das es mit `putchar` wieder schreibt.

Mit der Funktion `getchar` haben wir jetzt übrigens auch eine Möglichkeit, bei der Eingabe mehrerer Zahlen den Benutzer dazu zu zwingen, jede Zahl einzeln mit ENTER abzuschließen: Nachdem wir eine Zahl in der Zeile interpretiert haben, überspringen wir in einer Schleife alle Zeichen bis zum Zeilenende:

```
#include <stdio.h>

#define ANZAHL 10                /* Anzahl Wiederholungen */

int main(void)
{
    int i = ANZAHL, wert;        /* Variablendeklarationen */

    printf("Geben Sie die Zahlen ein. "
           "Bitte jeweils mit ENTER abschliessen!\n");

    while (i--) {                /* Schleife ! */
        scanf("%d", &wert);      /* eine Zahl lesen */
        while (getchar () != '\n') { /* Zeilenrest ignorieren */
        }
        printf("%d\n", wert);    /* gelesene Zahl schreiben */
    }
    return 0;
}
```

Quelltext 5.1: Kopieren von 10 Zahlen (Eine Zahl pro Zeile)

Dabei nutzen wir: `scanf` bleibt auf dem ersten Zeichen stehen, das *nicht* mehr interpretiert wurde, spätestens also auf dem Zeilenende-Zeichen. Das ist durchaus wesentlich für das Funktionieren des Programms: Die „Wegwerf“-Schleife wird ja mindestens einmal durchlaufen, holt also zumindest ein Zeichen – wenn sie kein Zeichen der „alten“ Zeile mehr finden würde (hier also zumindest das Zeilenende-Zeichen), würde sie über das Betriebssystem eine neue Eingabezeile anfordern – und das wäre hier ein vollständig unerwünschter Effekt.

5.4 Ein-/Ausgabe von Strings

Für die Ein-/Ausgabe von Strings stehen spezielle Funktionen zur Verfügung. Wir haben aber bereits alternative Möglichkeiten kennengelernt: Sowohl mit den Funktionen `scanf` und `printf` als auch mit den Funktionen `getchar` und `putchar` können wir Strings lesen und schreiben. Zur Erinnerung: Der entsprechende Formatbeschreiber für Ein- und Ausgabe ist `%s`.

Sehen wir uns zunächst die Ausgabe an. Sie ist mit beiden Möglichkeiten recht einfach:

```
#define LAENGE ??

char str[LAENGE];
int i;
/* ... Initialisierung von str ... */

printf ("%s", str);          /* Variante 1 */

i = 0;                       /* Variante 2 */
while (str[i]) {
    putchar(str[i]);
    i++;
}
```

Bei der Schleife sieht man direkt, dass die Ausgabe nur dann korrekt funktioniert, wenn der String korrekt durch ein Null-Zeichen abgeschlossen ist. Beim Aufruf von `printf` sieht man das zwar nicht, es gilt aber ebenso.

Die Eingabe ist, wenn man sie naiv programmiert, ähnlich einfach:

```
#define LAENGE ??

char str[LAENGE];
int i;

scanf ("%s", str);          /* Variante 1 */

i = 0;                       /* Variante 2 */
while ((str[i] = getchar ()) != '\n') {
    i++;
}
str[i] = '\0';
```

Ganz äquivalent sind beide Eingaben nicht:

- `scanf` überliest zunächst „white spaces“ und liest danach Zeichen, bis ein erneutes „white space“ gefunden wird, d.h. ein Leerzeichen, ein (horizontales) Tabulatorzeichen oder ein Zeilenende-Zeichen.

- In der Schleife wird auf jeden Fall von der aktuellen Position in der Zeile bis zum Zeilenende gelesen. Die Anweisung hinter der Schleife ersetzt das Zeilenende-Zeichen durch das Null-Zeichen.

„Naiv“ sind die beiden Realisierungen deshalb, weil in beiden Fällen nicht sichergestellt wird, dass der gelesene String in die Variable `str` hineinpasst.

Um den Aufruf von `scanf` entsprechend sicherer zu machen, benötigt man die erweiterten Möglichkeiten von Formatbeschreibern, wie sie in Kapitel 16 beschrieben werden. Die Schleife sollten wir jedoch geeignet modifizieren, was auch nicht übermäßig schwierig ist:

```
#define LAENGE ??

char str[LAENGE];
int i;

i = 0;
while (i < LAENGE && (str[i] = getchar ()) != '\n') {
    i++;
}
if (i == LAENGE) {
    printf("Eingegebener String zu lang!\n");
    return 1;
}
else {
    str[i] = '\0';
}
```

5.5 Klassifizierung von Zeichen

Neben `stdio.h` stellen auch die Standardheaderdateien `ctype.h` und `string.h` Standardfunktionen zur Verarbeitung von Zeichen und Strings zur Verfügung. Vielmehr stehen zwei weitere Headerdateien zur Verfügung:

- Die Headerdatei `ctype.h` definiert Funktionen zur Klassifizierung von Zeichen und zur Umwandlung zwischen Groß- und Kleinbuchstaben.
- Die Headerdatei `string.h` definiert Funktionen mit denen man
 - Strings kopieren, konkatenieren oder vergleichen,
 - in Strings nach Zeichen oder Strings suchen oder
 - verschiedene andere Dinge erledigen kann.

Der Header `ctype.h` soll an dieser Stelle kurz behandelt werden, weil zum einen der Umfang der Headerdatei nicht groß, zum anderen die Funktionen sehr grundlegend und einfach sind.

Die Funktionen `tolower` und `toupper` wandeln Groß- in Kleinbuchstaben bzw. Klein- in Großbuchstaben um; andere Zeichen bleiben unverändert. Das Argument ist jeweils das ggf. umzuwandelnde Zeichen in `int`-Darstellung, der Funktionswert das resultierende Zeichen in `int`-Darstellung.

Die übrigen Funktionen haben als Argument ebenfalls ein Zeichen in `int`-Darstellung; ihr Funktionswert ist ungleich oder gleich Null und kann entsprechend als *wahr* oder *falsch* interpretiert werden, je nachdem, ob das Zeichen zu einer bestimmten Gruppe von Zeichen gehört oder nicht. Eine Liste der Funktionen ist in Tabelle 5.1 zu finden.

In alle Funktionen (einschließlich `tolower` und `toupper`) darf man übrigens als Argument auch `EOF` übergeben – und erhält dann `EOF` als Funktionswert zurück.

Funktion	Beschreibung
<code>islower</code>	Kleinbuchstabe
<code>isupper</code>	Großbuchstabe
<code>isalpha</code>	Buchstabe, klein oder groß
<code>isdigit</code>	Dezimalziffer
<code>isxdigit</code>	Hexadezimalziffer
<code>isalnum</code>	Buchstabe oder Ziffer
<code>iscntrl</code>	Steuerzeichen (nicht druckbar)
<code>isgraph</code>	druckbares Zeichen (ohne Leerzeichen)
<code>isprint</code>	druckbares Zeichen (einschließlich Leerzeichen)
<code>ispunct</code>	druckbares Sonderzeichen (ohne Leerzeichen)
<code>isspace</code>	„white space“

Tabelle 5.1: Funktionen zur Klassifizierung von Zeichen

Als Beispiel betrachten wir einen Programmausschnitt, in dem eine Hexadezimalzahl als Zeichenfolge eingelesen und in die entsprechende interne Darstellung umgewandelt wird. Wir gehen davon aus, dass die eingegebene Zahl nicht größer als `UINT_MAX` ist. Die Interpretation soll stoppen, sobald das erste nicht interpretierbare Zeichen gefunden wird. Diese Aufgabe könnte man zwar auch mit `scanf` lösen – es ist aber durchaus nützlich, wenn man sich einmal Gedanken darüber macht, was bei der Ausführung einer Standardfunktion tatsächlich passiert.

```
#include <stdio.h>
#include <ctype.h>
...
int c;
unsigned int x;

x = 0;
while ( isxdigit( c = getchar() ) )
{
    x = x * 16;
    if ( isdigit(c) ) {
        x = x + c - '0';
    }
    else {
        x = x + toupper(c) - 'A' + 10;
    }
}
```

Diese Anweisungsfolge leistet das Verlangte, falls in dem verwendeten Zeichensatz die zehn Ziffern und die Buchstaben A bis F jeweils unmittelbar aufeinanderfolgen. Das ist die einzige Voraussetzung. Ansonsten ist dieses Programmstück ohne weiteres portierbar auf Implementationen, denen z.B. nicht der ASCII-Zeichensatz zugrunde liegt.

Die vielen Klammern sind übrigens alle wirklich nötig:

- Die Klammern um die Bedingung der `while`-Anweisung und um die Argumentliste (das Argument) von `isxdigit` sind offensichtlich notwendig.
- Beim Aufruf einer Funktion muss das Klammernpaar auch dann folgen, wenn die Funktion keine Parameter besitzt. (Wenn Sie diese Klammern wegließen, würde der Compiler also „maulen“ – allerdings nicht wegen fehlender Klammern, sondern wegen inkompatibler Typen; die Hintergründe können wir erst später behandeln.)

Ein sachlicher Mangel der Anweisungsfolge soll nicht unerwähnt bleiben: Es ist keinerlei Sicherung gegen eine Bereichüberschreitung eingebaut!

